

Algorithm Project

Birthday Cake Candles

Team Members: [Khaled Osama] | [Hamza Raef] | [Abdelrahman Sherif]
[Amr Sayed] | [Zyad Khaled] | [Youssef Ahmed] | [Amr Ayman]

Submission Date: May 14, 2026

Non-Recursive:

Pseudo-code:

```
Algorithm birthdayCakeCandles(candles, age)

maximum <- candles[0] //O(1)
count <- -1           //O(1)

For i <- 1 to age - 1 step 1      //O(n)
do{

    If candles[i] > maximum        //O(1)
        maximum <- candles[i]     //O(1)
        count <- -1               //O(1)

    Else if candles[i] == maximum  //O(1)
        count <- count + 1        //O(1)
}

Return count                     //O(1)
```

Analysis & Complexity:

Time Complexity:

- Each operation inside the loop is constant time $O(1)$
- $T(n)=O(n)$

Steps:

1. Read the number of candles.
2. Store candle heights in an array.
3. Compare each candle with the current maximum.
4. Update maximum and count when needed.
5. Print number of tallest candles.

Recursive:

Pseudo-code:

```
Algorithm findMaxAndCount(candles, age, index, maximum, count)

If index == age      //O(1)
    Return           //O(1)

If candles[index] > maximum    //O(1)
    maximum <- candles[index]  //O(1)
    count <- 1                 //O(1)

Else if candles[index] == maximum    //O(1)
    count <- count + 1              //O(1)

findMaxAndCount(candles, age, index + 1, maximum, count) //O(n)

Algorithm birthdayCakeCandles(candles, age)

maximum <- candles[0]    //O(1)
count <- 0               //O(1)

findMaxAndCount(candles, age, 0, maximum, count) //O(n)

Return count             //O(1)
```

Analysis & Complexity:

Steps:

1. Read the number of candles.
2. Store candle heights in an array.
3. Use recursion to compare candle heights.
4. Update maximum height and count.
5. Print number of tallest candles.

Time Complexity:

$T(n)=O(n)$

Compare:

-Performance: Non-Recursive is slightly faster than the Recursive

-Time Complexity: Both have the same complexity which is $O(n)$